Introduction

In this lab, we will perform penetration tests of a web application to determine if it has been securely designed.
DVWA (Damn Vulnerable Web Application!) is a PHP/MySQL web application. It is designed to be vulnerable to common attacks to allow security professionals to test their skills and tools and students and teachers to learn and understand web application security in a legal environment.

DVWA provides four levels of security: Low, Medium, High, and Impossible. Each security level requires different skills to perform exploits. The security levels reflect different levels of security that developers may code into their applications. In this lab, we will perform exploits against three of the security levels, Low, Medium, and High, allowing you to adjust your attacks to compromise them.

Part 1: Perform Reflective Cross Site Scripting Exploits

A Reflected XSS attack is one in which a malicious script is reflected off a web server to the user's browser. The script is activated through a link that the victim clicks. This will send a request to the website that has a vulnerability that enables execution of the malicious script
.

Step 1: Log into DVWA.

- From the Kali Linux VM, open a browser and navigate to the DVWA application.
- Enter the default dvwa localhost into the browser http://127.0.0.1/dvwa/setup.php. Note, remember to first set up DVWA database
- At the login prompt, enter the credentials: admin/password.
- See below my DVWA homepage below

Step 2: Perform a Reflected XSS attack at Low security level.

- Select DVWA Security in the left menu and select Low in the Security Level dropdown. Click Submit.
- Select XSS (Reflected) from the left menu.
- Type the string Reflected_Test in the What's your name? box and click Submit.

You will see the message Hello Reflected_Test appear.

- Enter CTRL+U on the keyboard to view the source code of the page.
- Search for the string Hello Reflected_Test by entering CTRL+F to open a search box.

The presence of the string in the page source HTML indicates that values entered in a user response text field are inserted into the source code for the page. This indicates to an attacker that the page may be vulnerable to reflected XSS attacks.

- Close the source code window and return to the Reflected XSS Vulnerability page.
- Enter the following payload in the What’s your name? box and click Submit.

<script>alert("You are hacked!")</script>

An alert popup box will appear with the words **You are hacked!**. This means the site is vulnerable to Reflected XSS attacks and we have successfully exploited the vulnerability.

- Select and copy the URL for the compromised page. Open a new browser tab and paste the URL into the URL field and press Enter.

You should see the same web page appear displaying the **You are hacked!** popup box. This means that if a user opens the URL a malicious script will execute. The alert box is used to simulate a malicious script in this lab.

- Below are the step by step screenshots of step 2: Perform a Reflected XSS attack at Low security level.

http://127.0.0.1/dvwa/security.php

ools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
   Prior to DVWA v1.9, this level was known as 'high'.

Low Submit

Additional Tools

- View Broken Access Control Logs - View access logs for the Broken Access Control vulnerability

e pointer outside or press Ctrl+Alt.

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name? Submit

Hello Reflected_Test

More Information

- https://owasp.org/www-community/attacks/xss/
- https://owasp.org/www-community/xss-filter-evasion-cheatsheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- https://www.cgisecurity.com/xss-faq.html
- https://www.scriptalert1.com/

use pointer outside or press Ctrl+Alt.

Kali Linux | 404 Not Found | Problem loading page | Vulnerability: Reflected C | http://127.0.0.1/dvwa/vulner

Not Secure view-source:http://127.0.0.1/dvwa/vulnerabilities/xss_r/?name=+Reflected_Test+#

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

```
1 <!DOCTYPE html>
2
3 <html lang="en-GB">
4
5     <head>
6         <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
7
8         <title>Vulnerability: Reflected Cross Site Scripting (XSS) :: Damn Vulnerable Web Application (DVWA)</title>
9
10        <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
11
12        <link rel="icon" type="\image/ico" href="../../favicon.ico" />
13
14        <script type="text/javascript" src="../../dvwa/js/dvwaPage.js"></script>
15
16    </head>
17
18    <body class="home light">
19        <div id="container">
20
21            <div id="header">
22
23                <img src="../../dvwa/images/logo.png" alt="Damn Vulnerable Web Application" />
24                <a href="#" onclick="javascript:toggleTheme();" class="theme-icon" title="Toggle theme between light and dark.">
25                    <img src="../../dvwa/images/theme-light-dark.png" alt="Damn Vulnerable Web Application" />
26                </a>
27            </div>
28
29            <div id="main_menu">
30
31                <div id="main_menu_padded">
32                <ul class="menuBlocks"><li class=""><a href="../../.">Home</a></li>
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

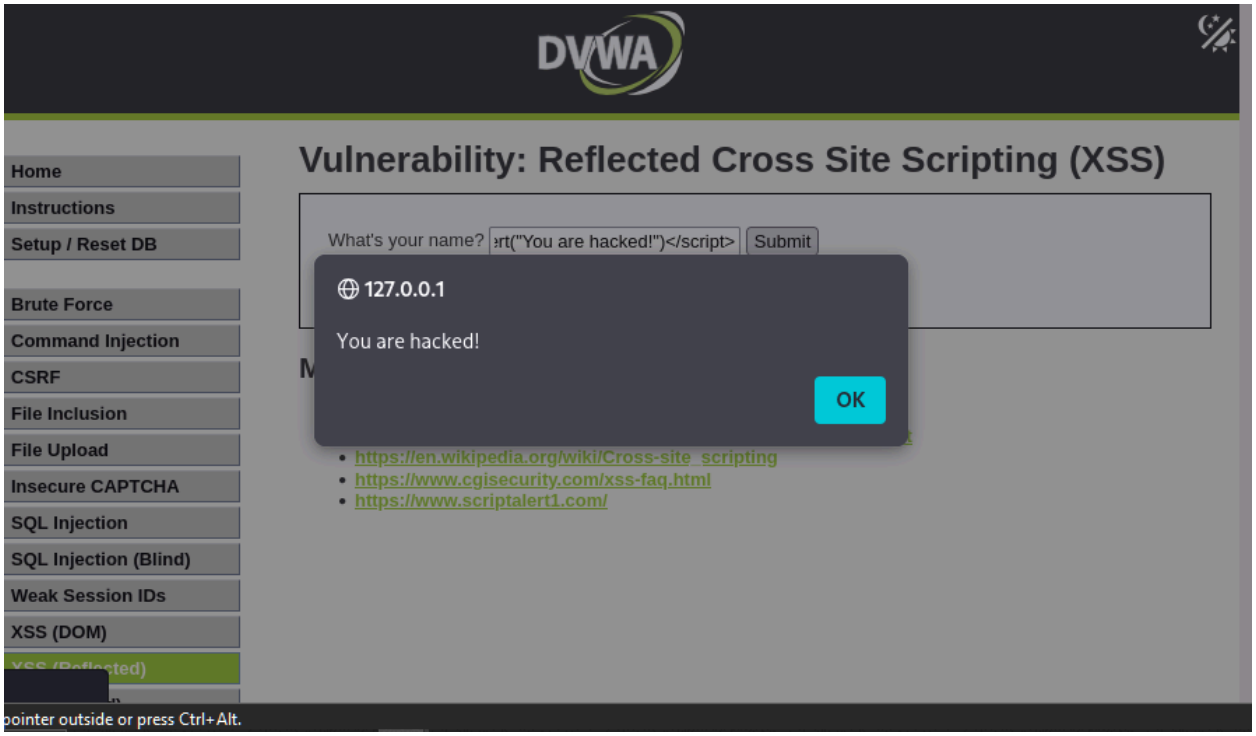
Not Secure view-source:http://127.0.0.1/dvwa/vulnerabilities/xss_r/?name=+Reflected_Test+#

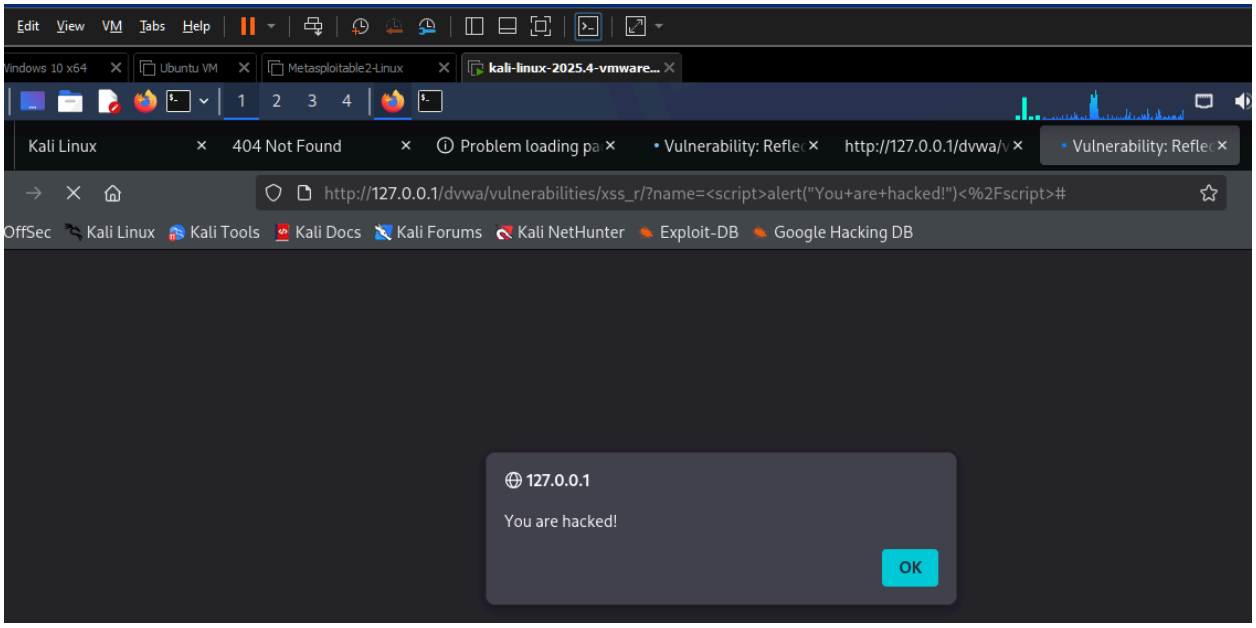
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

```
62            <div id="main_body">
63
64
65 <div class="body_padded">
66     <h1>Vulnerability: Reflected Cross Site Scripting (XSS)</h1>
67
68     <div class="vulnerable_code_area">
69         <form name="XSS" action="#" method="GET">
70             <p>
71                 What's your name?
72                 <input type="text" name="name">
73                 <input type="submit" value="Submit">
74             </p>
75
76         </form>
77         <pre>Hello  Reflected_Test</pre>
78     </div>
79
80     <h2>More Information</h2>
81     <ul>
82         <li><a href="https://owasp.org/www-community/attacks/xss/" target="_blank">https://owasp.org/www-community/attacks/xss/</a></li>
83         <li><a href="https://owasp.org/www-community/xss-filter-evasion-cheatsheet" target="_blank">https://owasp.org/www-community/xss-filter-evasion-cheatsheet</a></li>
84         <li><a href="https://en.wikipedia.org/wiki/Cross-site_scripting" target="_blank">https://en.wikipedia.org/wiki/Cross-site_scripting</a></li>
85         <li><a href="https://www.cgisecurity.com/xss-faq.html" target="_blank">https://www.cgisecurity.com/xss-faq.html</a></li>
86         <li><a href="https://www.scriptalert1.com/" target="_blank">https://www.scriptalert1.com/</a></li>
87     </ul>
88 </div>
89
90                 <br /><br />
91
92
```

Hello Reflected_Test | Highlight All | Match Case | Match Diacritics | Whole Words | 1 of 1 match

To return to your computer, move the mouse pointer outside or press Ctrl+Alt.





Step 3: Perform a Reflected XSS attack at Medium security level.

We will attempt the same attack, but this time the security level of the Web site will be Medium

- Select DVWA Security in the left menu and select Medium in the Security Level dropdown. Click Submit.
- Select XSS (Reflected) in the left menu.
- Again, enter the following payload in the What's your name? box and click Submit.

```
<script>alert("You are hacked!")</script>
```

You will see a Hello response, but this time no pop up will appear. This indicates that the script did not execute. Note that the script is displayed as literal text.

We can analyze the code in the backend of the web site to investigate the reason.

- Click the View Source button on the bottom right of the page and review the PHP code.

Note: On a real web server, we would not have access to this backend source code, but here on DVWS we do.

- Note this line:

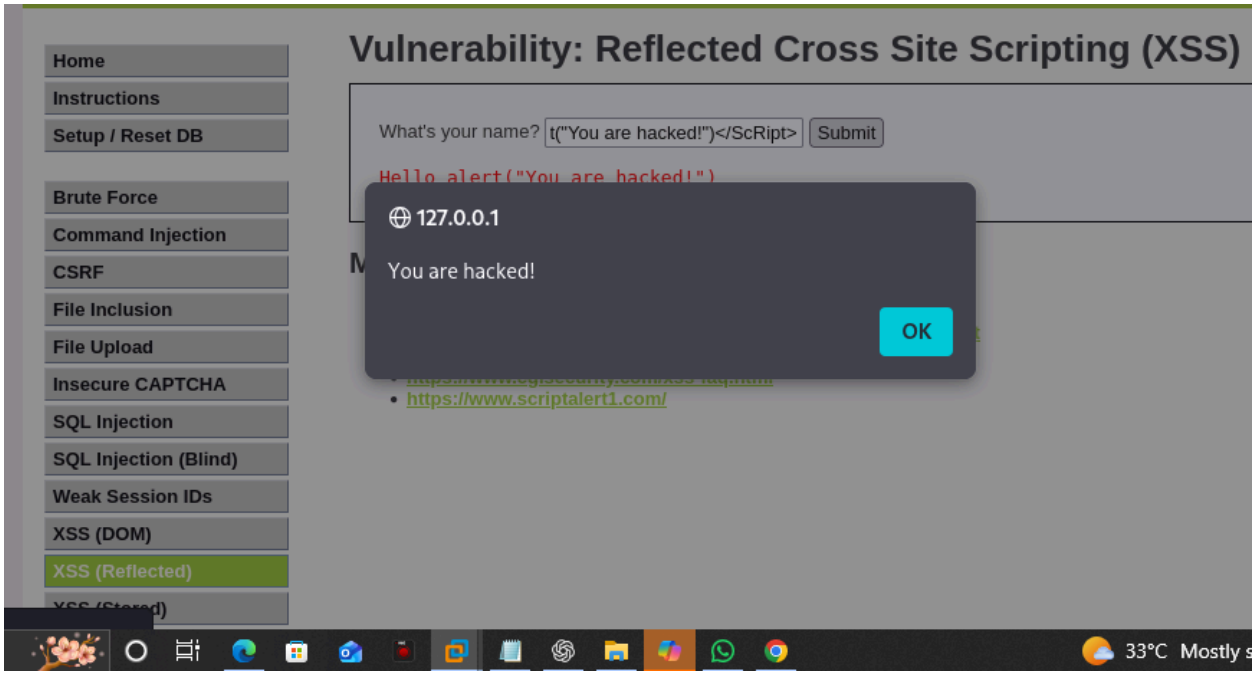
```
$name = str_replace ( '<script>', '',  $_GET[ 'name' ] );
```

This source code creates a filter, with **str_replace()** function, that removes the **<script>** tag in our payload and replaces it with a null value. This renders the payload script ineffective, so the attack fails, and no popup window is displayed. Because this script is only filtering out **<script>** in lower case, we can try and get around the filter by using a different tag in the payload. We will use **<ScRipt>**.

- Close the source code window and return to the Reflected XSS Vulnerability page.
- Enter the following payload in the What's your name? box and click Submit.

```
<ScRipt>alert("You are hacked!")</ScRipt>
```

- Below the screenshots of Medium in the Security Level, the outcome <ScRipt>alert("You are hacked!")</ScRipt> payload in the what's your name box



Step 4: Perform a Reflective XSS attack at High security level.

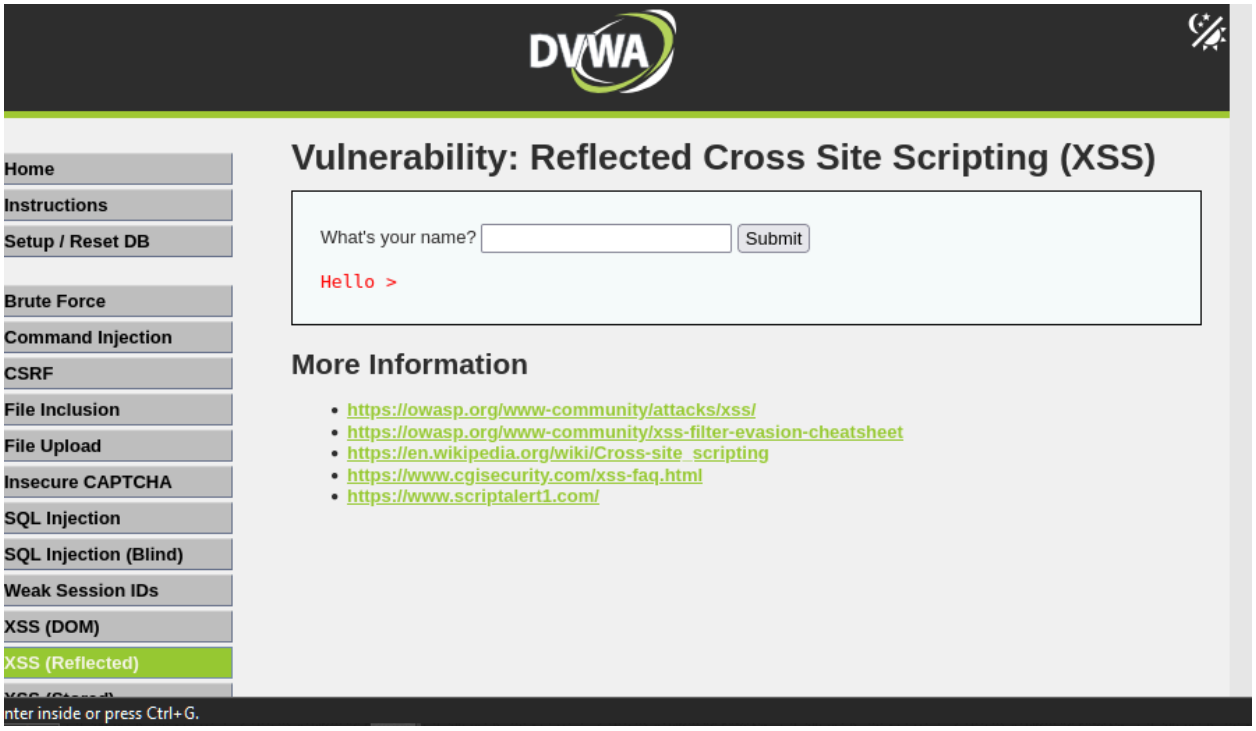
The same attack will be attempted, but this time the security level of the website will be High

- Select DVWA Security in the left menu and select High in the Security Level dropdown. Click Submit

- Select XSS (Reflected) in the left menu.
- Enter the following payload in the What's your name? box and click Submit. (Note the use of underscores to replace spaces.)

<ScRipt>alert("You_are_hacked!")</ScRipt>

There is a Hello message and no alert pop up box. Again, we can analyze the backend source code to investigate.



- Click the View Source button and review the PHP code.

Note the following line:

```
$name = preg_replace( '/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/i', '', $_GET[ 'name' ] );
```

In this code, the developer used a regular expression to replace any form of the **<script>** tag, no matter what case of the characters is used, with a null value.

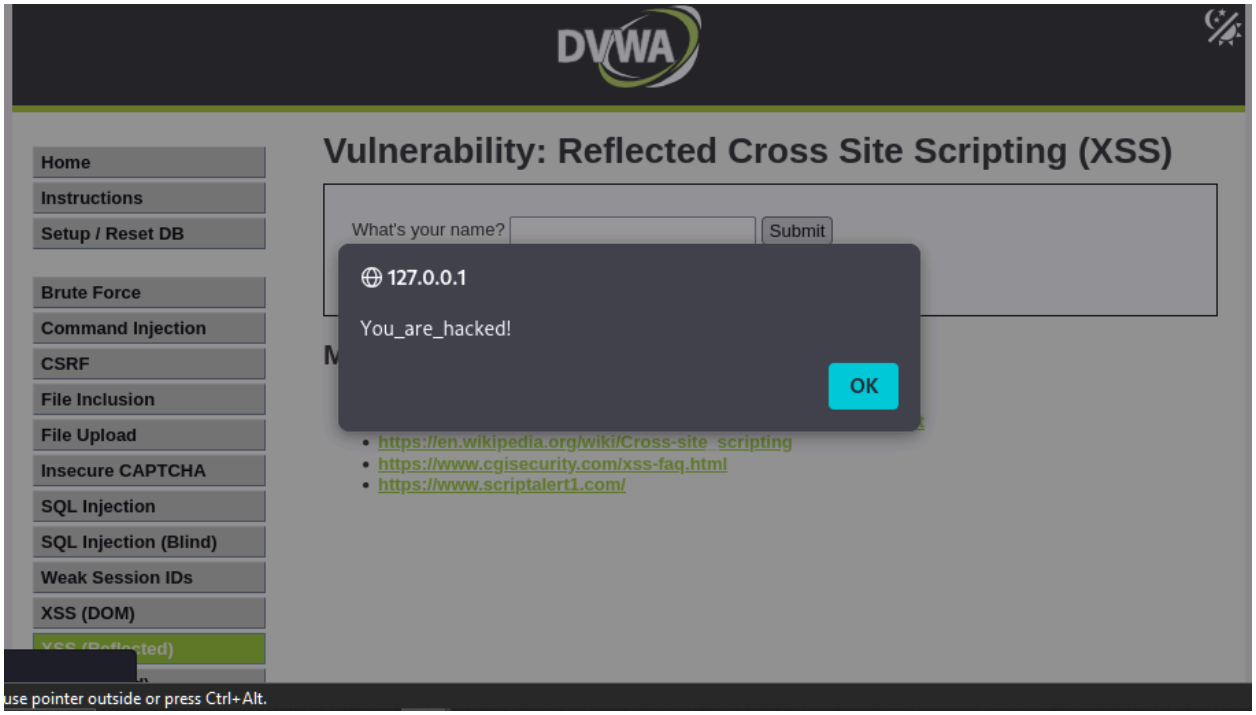
- To bypass this filter, we must use another HTML tag instead of **<script>** to attack the site.

Close the source code window and return to the Reflected XSS Vulnerability page.

- Enter the following payload in the What's your name? box and click Submit. (Note the use of underscores to replace spaces.)

<img src=x onerror=alert("You_are_hacked!")>

The XSS popup box will appear this time. We successfully bypassed the filter and exploited the Reflected XSS vulnerability in DVWA at High level security.



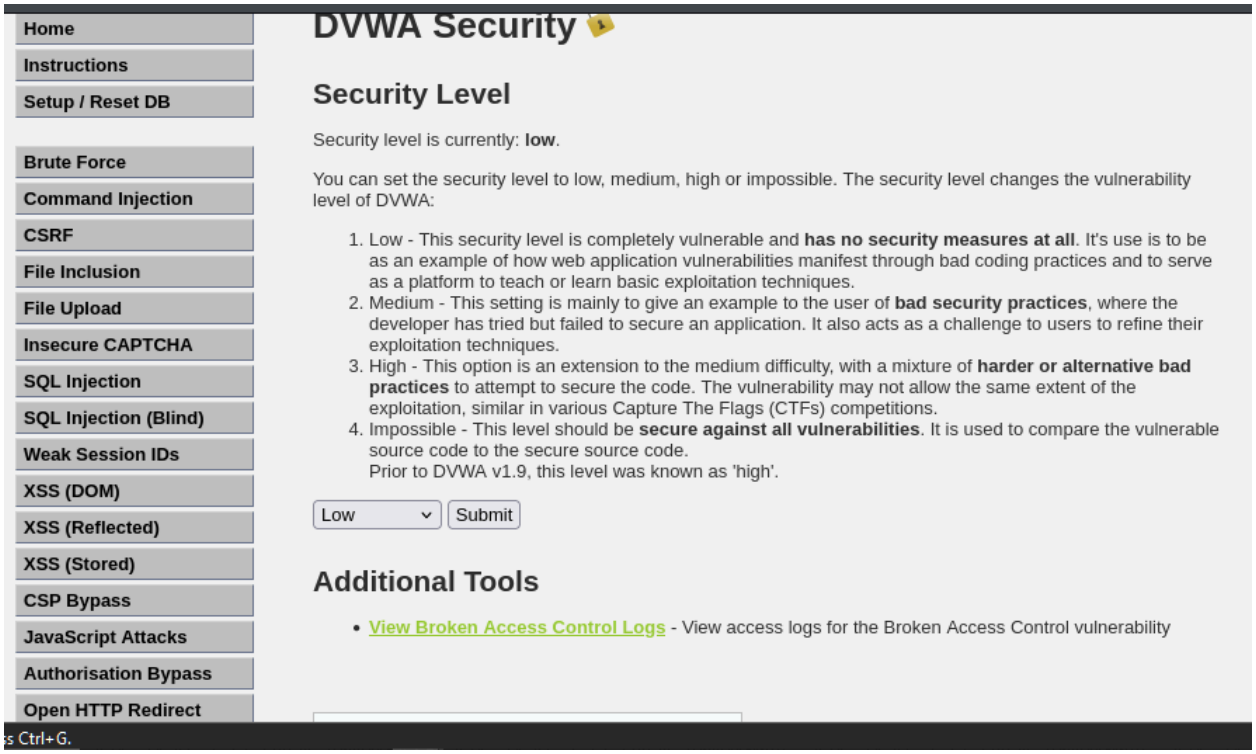
Part 2: Perform Stored Cross Site Scripting Exploits

With the stored XSS exploit, you enter a malicious script through user input and the script is stored on the target server in a message forum, database, visitor log, or comment field. When a user visits the target, the server exposes the user to the malicious code.

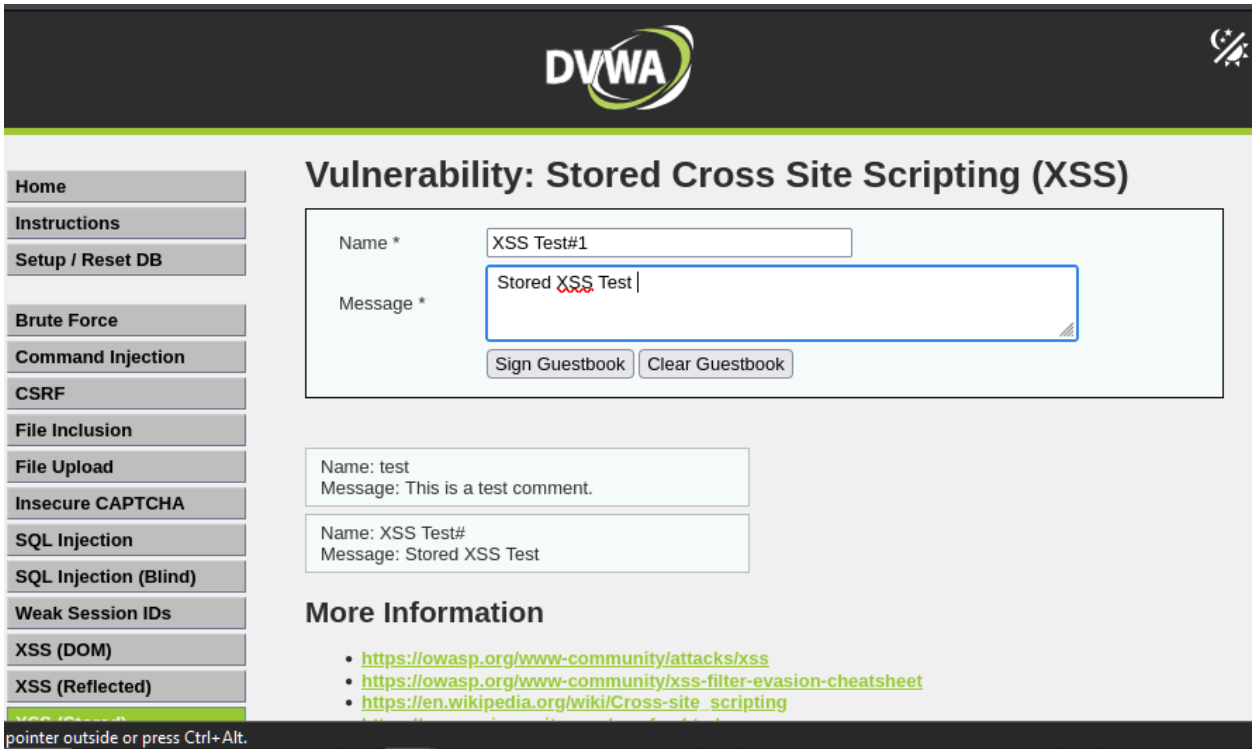
Step 1: Perform a Stored XSS attack at Low security level.

Exploiting stored XSS at low level security is easy because there are no security measures in place. You can simply submit a **<script>** to achieve the exploit.

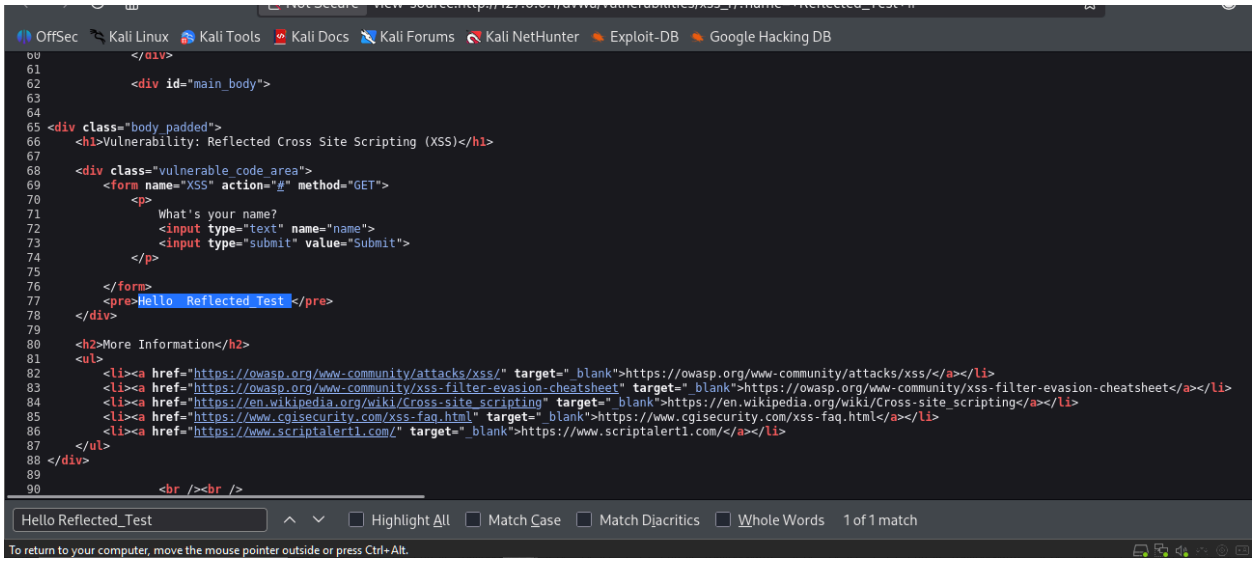
- Select DVWA Security in the left menu and select Low in the Security Level dropdown. Click Submit.



- Select XSS (Stored) in the left menu.
- Type the string **XSS Test#1** in the Name* field and type **Stored XSS Test** in the Message * field. click Sign Guestbook.



- Enter CTRL+U on the keyboard to view the page source code. Enter CTRL+F to search for the **Test#1** and **Stored XSS Test strings**.

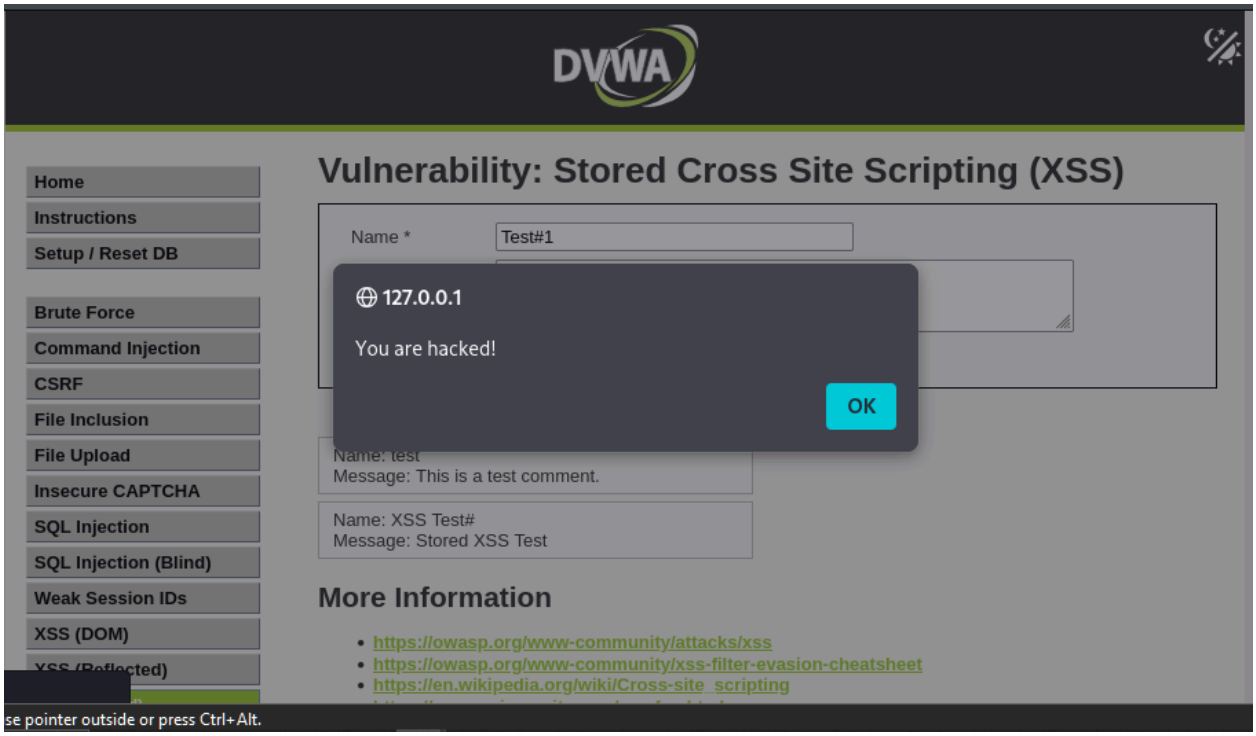


Both strings, **Test#1** and **Stored XSS Test,** will be in the page source code indicating that the two input fields may be vulnerable to a Stored XSS attack.

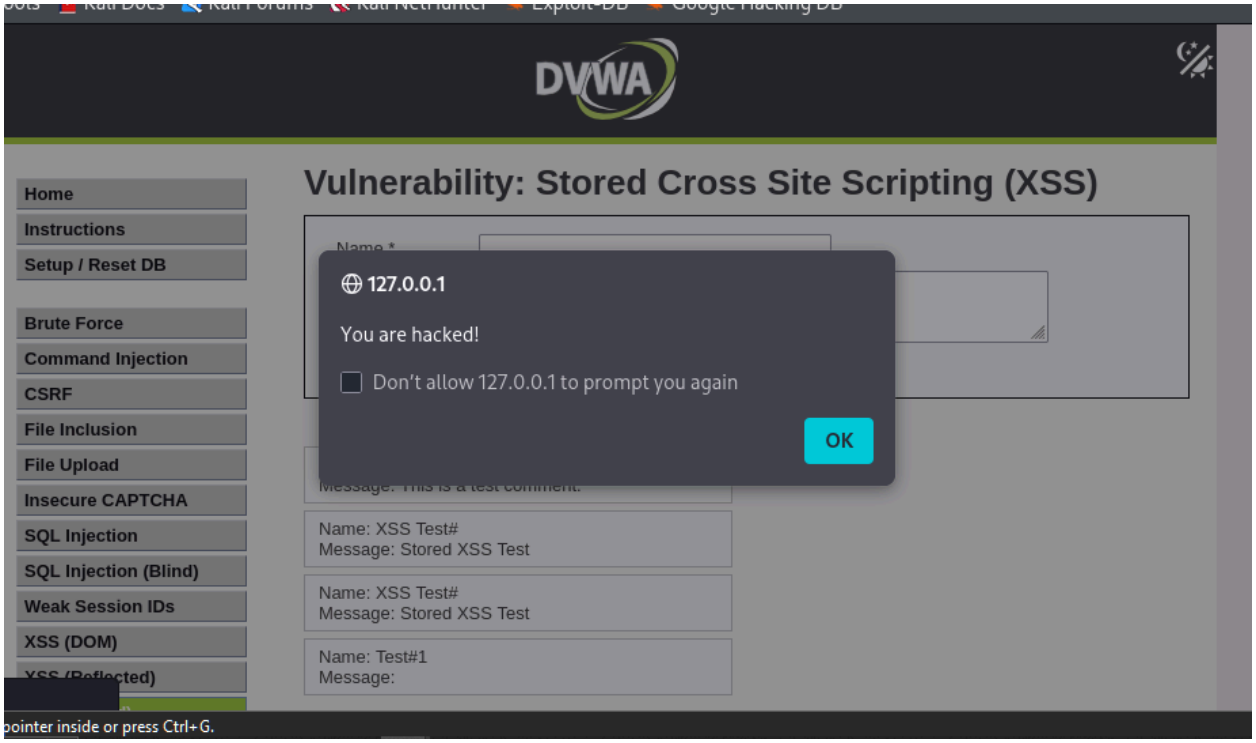
- Close the source code window and return to the Stored XSS Vulnerability page.
- Enter **Test#1** in the Name * box and enter the following payload in the Message * field and click Sign Guestbook.

  <script>alert("You are hacked!")</script>

An XSS alert popup box will appear with the words **You are hacked!.** This means the site was vulnerable to stored XSS attacks and we have successfully exploited the vulnerability.



- Refresh the page. If alerted, click Resend to display the page. The XSS alert popup box will appear again.



Because the XSS payload is stored in the guestbook, the alert popup box will appear each time the page is refreshed.

- Before proceeding to the next step, clear the XSS payload from the page. Click Setup / Reset DB in the left menu then click Create / Reset Database.

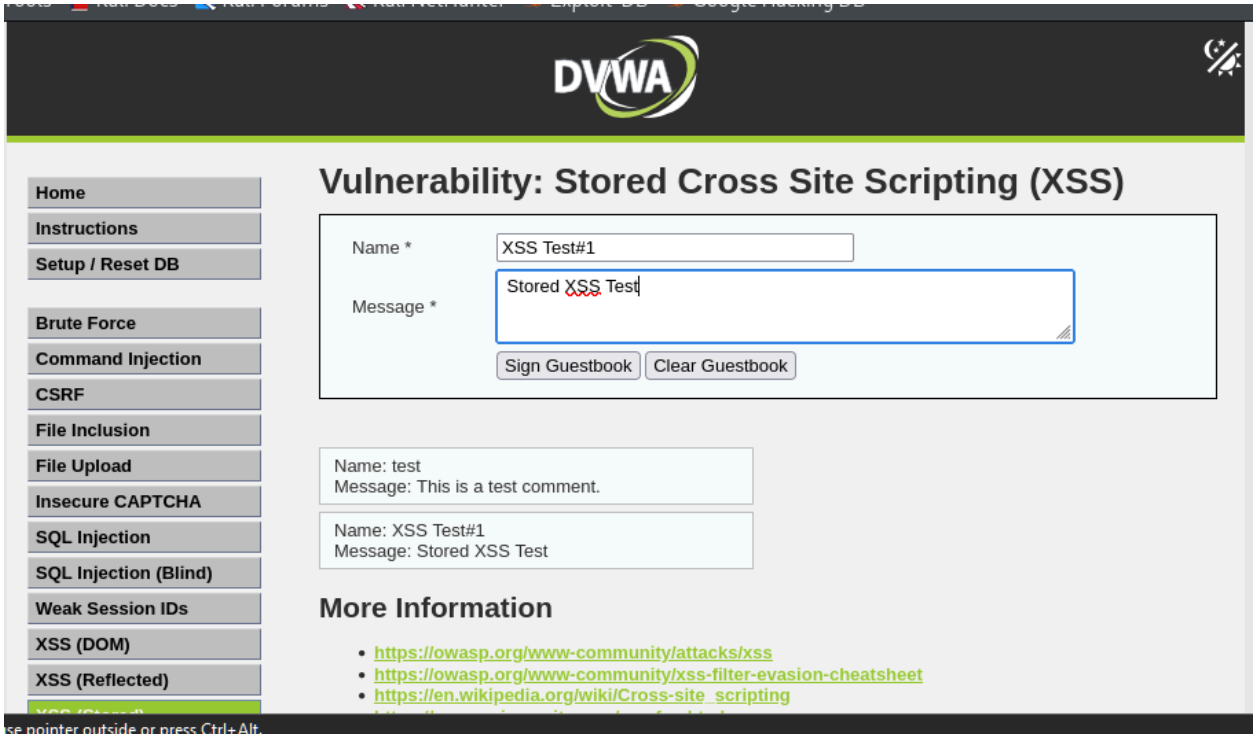
Step 2: Perform a Stored XSS attack at Medium security level

The same attack will be attempted, but this time the security level of the Web site will be changed to Medium.

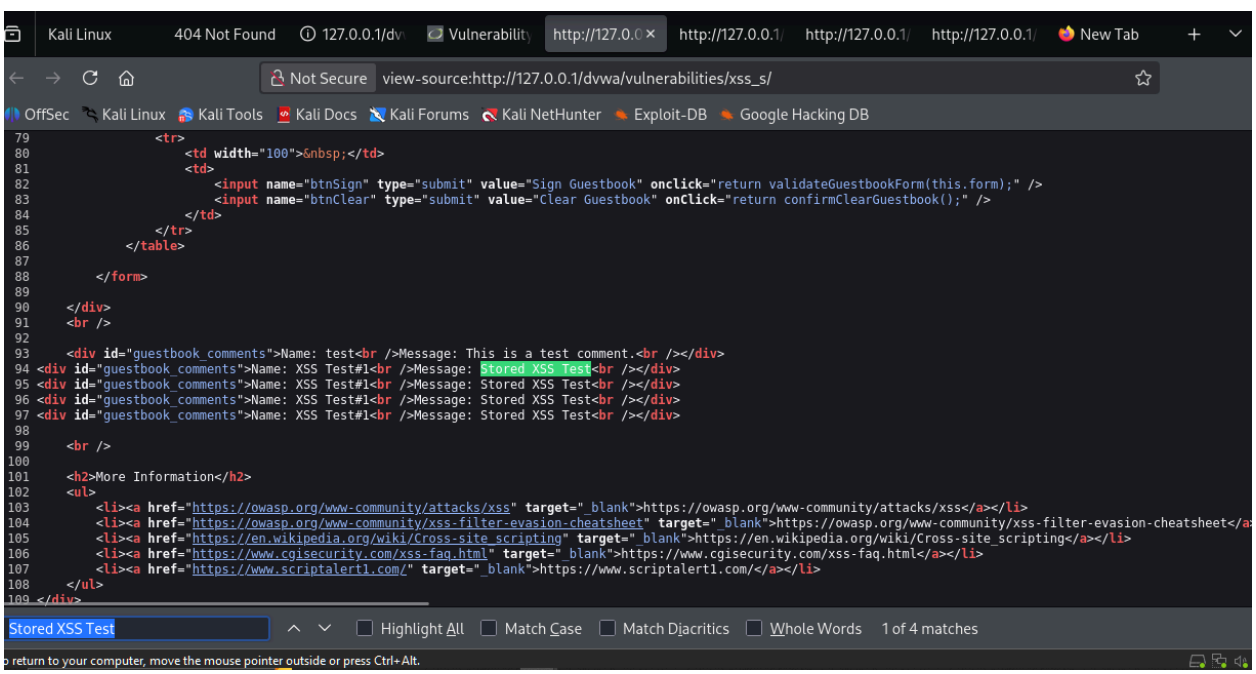
Exploiting reflected XSS at medium level security is not difficult. Using **<script>** will be rejected but changing it to a different case such as **<ScRipt>** will bypass the security and achieve the exploit.

- Select DVWA Security in the left menu and select Medium in the Security Level options dropdown. Click Submit.

- Select XSS (Stored) in the left menu.
- Type the string **XSS Test#1** in the Name* field and type **Stored XSS Test** in the Message * field. click Sign Guestbook.

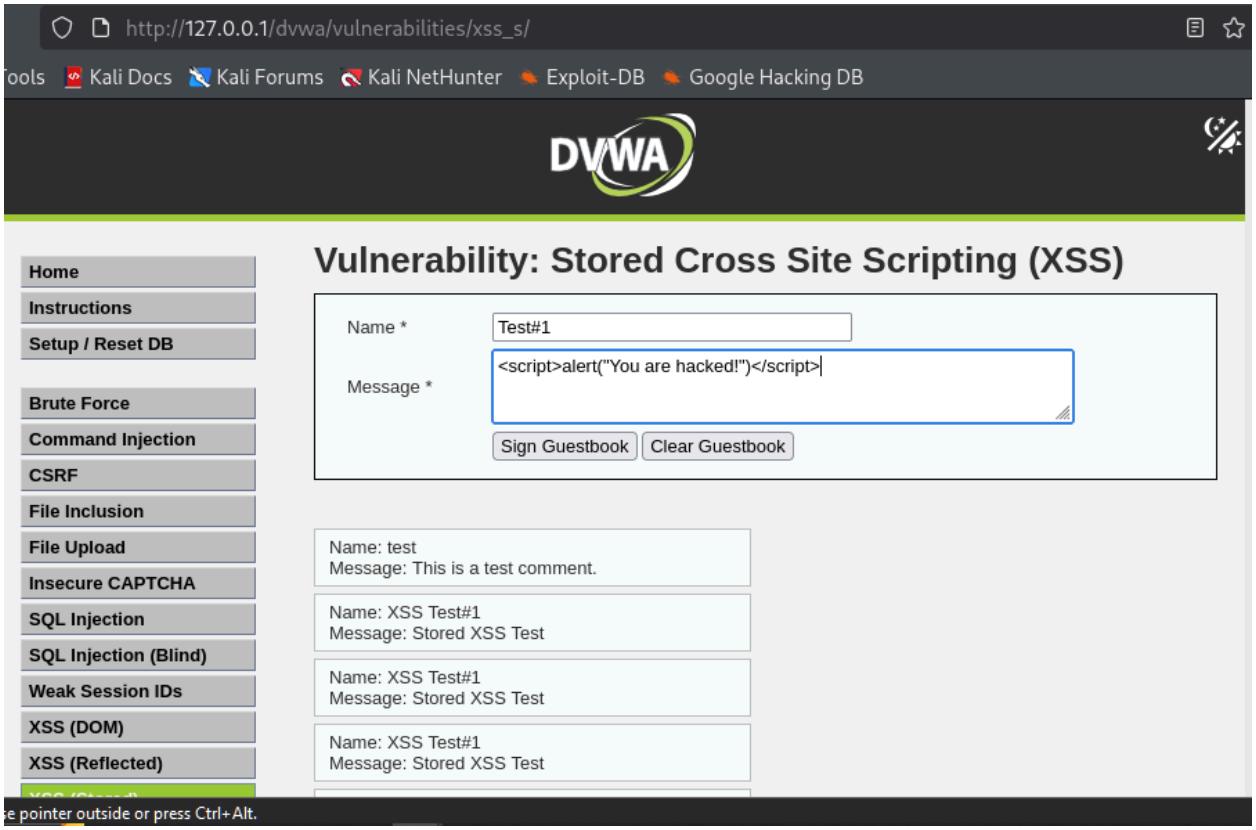


- Enter **CTRL+U** on the keyboard to view the page source code. Enter **CTRL+F** to search for the **Test#1** and **Stored XSS Test** strings.
- Both strings will be in the page source code indicating that the two input fields may be vulnerable to a Stored XSS attack.



- Close the source code window and return to the Vulnerability page.
- Enter **Test#1** in the Name * box and enter the following payload in the Message * box and click Sign Guestbook.

<script>alert("You are hacked!")</script>



No popup box should appear. Refreshing the page should not cause the alert popup box to appear either.

This means that there is code in the backend that is sanitizing the user input from the Message * field to prevent scripts from being submitted. You can see the modified input in the last rectangle message box below the input fields.

- Click the View Source button and review the PHP source code and investigate.

You will see two blocks of code with the word Sanitize. The first block of code, under // Sanitize message input, contains two PHP functions for performing input sanitization. The strip_tags() function removes all html tags from the message field before storing them in the database. The

htmlspecialchars() function converts all special characters into equivalent HTML entities so they are not reflected back in the browser.
The second block of code, under **// Sanitize name input**, performs input sanitation on the Name * field. It contains the str_replace() function which replaces any occurrence of the <script> tag with a null value. This disables the script completely.

We can attempt to bypass the security on the Name * field by using some other payload that does not contain <script> tags.

- Close the source code window.
- Before entering any payload into the Name * field, the max character length restriction of 10 characters on the field must be increased. This is a client-side setting so it is easy to change with the following steps:

Right-click in the Name * field and select Inspect. This opens the Web Developer Tools window and displays the page source code.

- Find and double-click maxlength in the page source and change it from 10 to 100. The maxlength property is inside the <input> tag for the text field
- Press Enter on the keyboard to apply the changes.

- Close the Web Developer's Tools Window.

With the maxlength restriction changed, the XSS payload can now be entered into the Name * field.
Note: Changing the maxlength parameter does not persist. If you refresh the page, for example, the setting needs to be changed again.

- Return to the Vulnerability page and enter the following payload in the Name * field.

<ScRipt>alert("You are hacked!")</ScRipt>

- In the Message * field you can type any text you like and then click Sign Guestbook.

An XSS alert popup box will appear with the words You are hacked!.

Because the XSS payload is stored in the guestbook, the alert popup box will appear each time the page is refreshed or each time other users visit the page.

The popup confirms you have successfully exploited Stored XSS vulnerability at the Medium level of security.

- Before proceeding to the next step, clear the XSS payload from the page. Click Setup / Reset DB in the left menu then click Create / Reset Database.

Step 3: Perform a Stored XSS attack at High security level

you will attempt the same attack, but this time the security level of the Web site will be set to High.

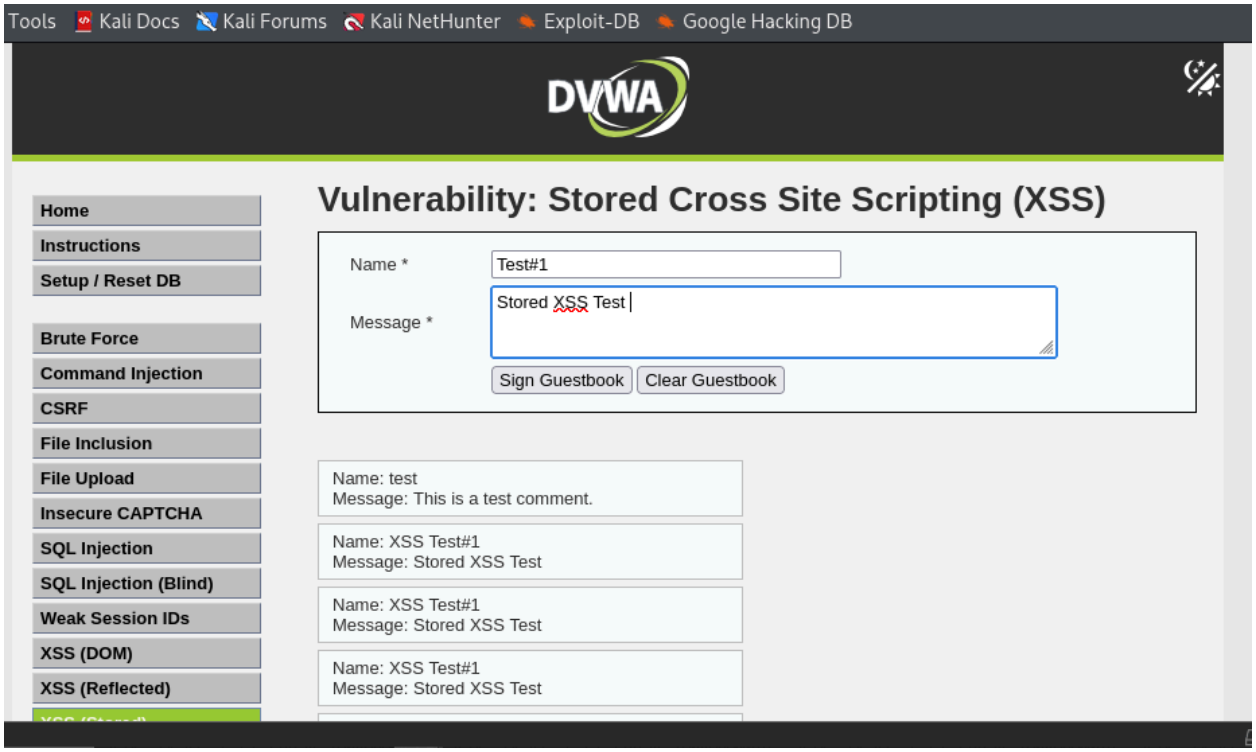
At high level the security code whitelists <scripts> and all else is rejected. However, other tags, such as svg will work

Select DVWA Security in the left menu and select High in the Security Level dropdown. Click Submit.

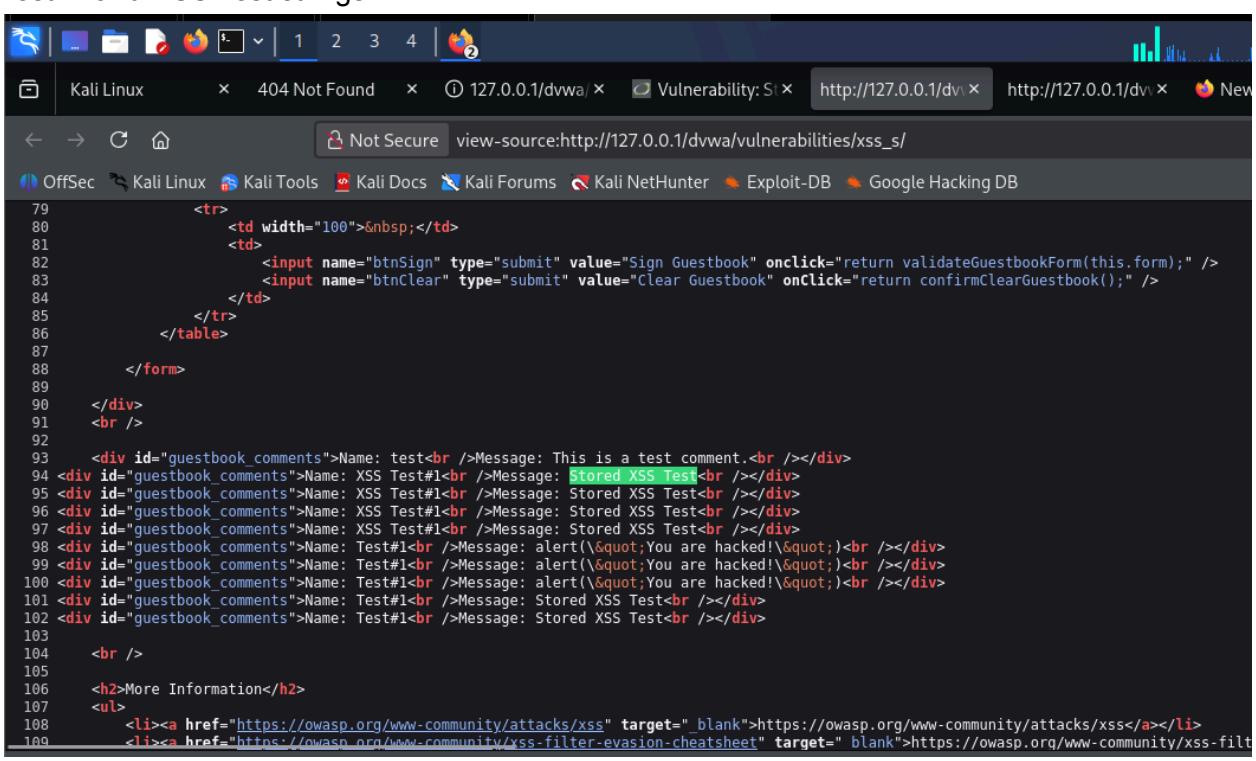
Select XSS (Stored) in the left menu.

Type the string **Test#1** in the Name* field and type **Stored XSS Test** in the Message * field.

Click Sign Guestbook.



Enter CTRL+U on the keyboard to view the page source code. Enter CTRL+F to search for the Test#1 and XSS Test strings.



Both Test#1 and XSS Test should be in the page source code indicating that the two input fields may be vulnerable to a Stored XSS attack.

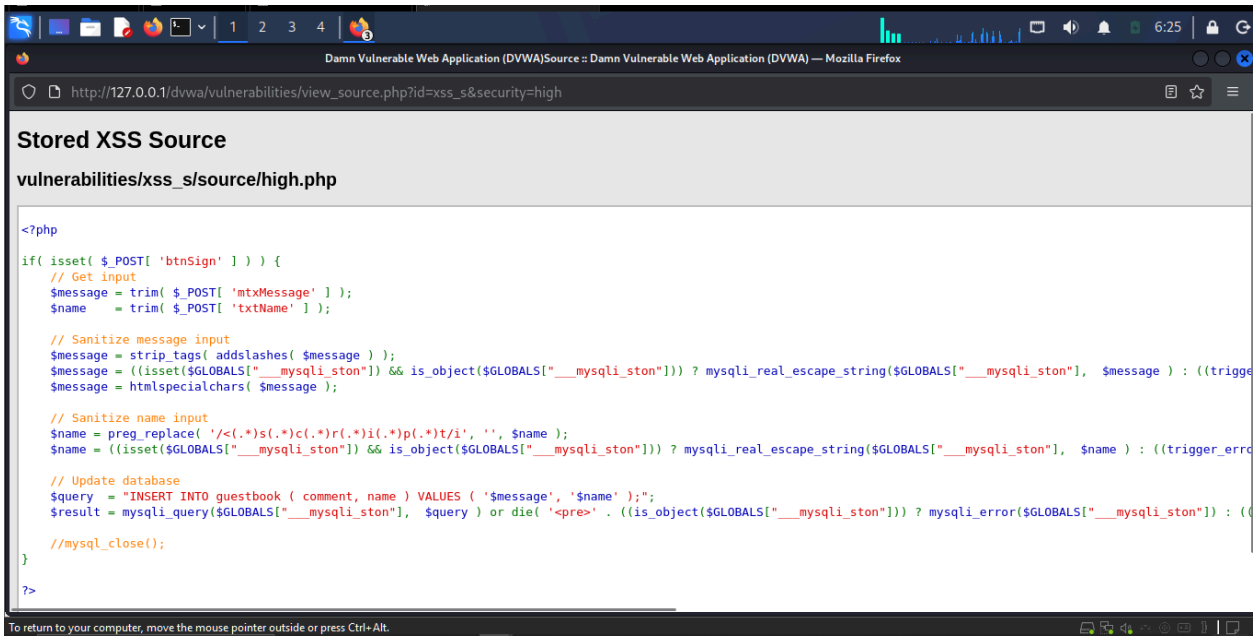
- Close the page source code tab and return to the Vulnerability page.
- Enter Test#1 in the Name * box and enter the following payload in the Message * box and click Sign Guestbook.

<ScRipt>alert("You are hacked!")</ScRipt>

No popup box will appear. Refreshing the page will not cause the alert popup box to appear either.

This means that there is code in the site backend that is sanitizing the user input from the Message * field.

- Click the View Source button and review the PHP source code and investigate.



You will see two blocks of code. As before with the Medium security, the first block of code, under **// Sanitize message input**, contains two php functions for performing input sanitization. The **strip_tags()** function removes all html tags from the message field before storing them in the database. The **htmlspecialchars()** function converts all special characters into equivalent HTML characters so they are not reflected back in the browser.

The second block of code, under **// Sanitize name input**, is performing input sanitation on the Name * field. It contains the **preg_replace()** function. This function uses a regular expression to replace any occurrence of the **<script>** tag, regardless of character case, with a null value.

We can attempt to bypass the security on the Name * field by using some other payload that does not contain <script> tags.

- Before entering any payload into the Name * field, it will be necessary to change the max character length restriction on the field, as was done above.

With the maxlength restriction changed, the XSS payload can now be entered into the Name * field.

- Return to the Vulnerability page and enter the following payload in the Name * field. (Note the use of underscores to replace spaces.)

<svg onload=alert("You_are_hacked!")>

- In the Message * field, you can type any text you like and then click Sign Guestbook.

An XSS alert popup box will appear with the message **"You_are_hacked!".**

Because the XSS payload is stored in the guestbook, the alert popup box will appear each time the page is refreshed.
The popup confirms you have successfully exploited a Stored XSS vulnerability at High security level.

Conclusion

Through this lab, we successfully demonstrated how Cross Site Scripting (XSS) vulnerabilities can be exploited across different security levels in DVWA. By performing reflective XSS attacks, I gained practical insight into how insecure coding practices expose applications to malicious inputs and how varying levels of security implementation affect the difficulty of exploitation. This exercise reinforced the importance of secure development practices, proper input validation, and continuous testing in safeguarding web applications. Ultimately, the project highlights that understanding offensive techniques is essential for building stronger defensive strategies in cybersecurity.